

Remove This Dependency

What we built, in plain terms

The one-sentence version

Benchmarks that grade code changes by "do the tests still pass" are trusting an umpire who is often looking the other way. We measured how often, built a benchmark for a task that exposes it, and built a grader that does not depend on tests alone.

Part 1 — The task nobody measures

Every Python project declares its dependencies in a file. That file is a *claim* about what the code needs, and claims drift. Developers delete the last use of a library and forget the declaration. Over half of declared dependencies in the PyPI ecosystem are unnecessary, and the single largest cause is refactoring that left the declaration behind.

So there is a real task: **remove a dependency the code actually uses** — rewrite the code so it no longer needs the library, and keep the project working.

Nobody benchmarks it.

- Dependency-inference benchmarks ask a model to *restore* declarations.
- Debloating tools remove dependencies that are *already unused* — no rewriting.
- Migration benchmarks swap one library for another, which is a different job.

Removal-with-rewrite sits in the gap. That is the task we build for.

Part 2 — The umpire is blind

Benchmarks in this area grade by execution: apply the change, run the project's own test suite, and if it passes, the change was correct.

The benchmark we build on states this outright — "*whether the tests pass is the most direct and reliable indicator of correctness*" — and its own limitations section raises the possibility that tests might not cover everything, only to set it aside.

We measured it.

A warning sign before we even start

Before testing anything of ours, we re-ran the benchmark's *own* published answers — the correct solutions it ships — through its own test suites.

51 of 96 still pass.

Nearly half no longer work. Not because the answers were wrong when written, but because the world moved: packages released new versions, services changed, pinned things drifted. We therefore only ever measure against repositories whose correct answer still works today. Every number below is conditioned that way.

What we did

Take every dependency, of every repository whose tests pass in our harness. Delete the declaration. Make the package genuinely unavailable to that repository's own code. Re-run the real test suite in a container.

622 dependencies. 80 repositories. Every one accounted for.

What happened

In **239** cases the tests passed anyway. The library was gone and nothing noticed.

But that single number would be misleading, so we opened it up:

What was really going on	How many
The package was never used anywhere — the declaration was simply wrong	172
Used, but not through an import (a plugin, a command-line tool)	13
The code genuinely uses it, and no test ever touches that path	49
The project is written to work without it	5

49 is the real finding. The code needs the library on some path, and the test suite has nothing to say about that path. Delete it and the umpire calls it safe.

The other rows are findings too, just different ones. 172 wrong declarations is a measurement of how noisy these files are. And the 13 "used without importing" cases are a warning: any tool that reasons about imports — including ours — cannot see them.

How common is it?

Not uniform, and we do not pretend otherwise. One repository declares 172 dependencies by itself, so a single pooled average would really be measuring that repository. Reported properly:

- In a typical repository, **1 in 5** declared dependencies can be deleted without the tests noticing.
- **35 of 51** repositories have at least one such dependency.
- Five repositories have none at all — so this tracks how thoroughly a project is tested, rather than being universal.

Part 3 — A benchmark where the umpire can see

The blind cases are useless as test questions: if the tests do not notice the library leaving, they cannot tell whether an agent removed it properly.

So the benchmark keeps only the cases where the tests *do* notice.

335 tasks, across 74 real repositories.

Each one is: *this project uses this library, here on these lines; remove it and keep the tests green*. Four difficulty tiers, by how much of the codebase touches the library — from one file to dozens.

Two things make it trustworthy:

Nothing is hidden. All 622 candidates are published with their outcome — 335 kept, 239 where the tests were blind, 48 excluded with the reason written down. Surveys of benchmark quality find that almost nobody does this.

A failure has to be the right failure. A task counts only if the test suite breaks *because this repository needs this library* — not because some unrelated package did, not because a linter complained about something else. We check the error came from the project's own code.

Part 4 — A better umpire

Now the harder half. Suppose an agent attempts one of these tasks and the tests pass. Has it actually removed the dependency?

Not necessarily. There are at least eight ways to fake it:

- move the declaration somewhere the checker does not look
- paste the library's source code into the project
- write a fake module with the same name that returns plausible nonsense
- delete or skip the tests that would have noticed
- swap in a different library
- ...and so on

We built **617 of these fakes** — every family, across real repositories — and ran them through a stack of checks.

What the checks catch

The trick	Caught
Hiding the declaration	77 of 77
Swapping in another library	73 of 73
Weakening the tests	78 of 80
Pasting in the library's code	48 of 67
Fake module with the same name	65 of 80
An honest removal (the control)	left alone, 79 of 80

That last row matters as much as the others. A checker that rejects everything would look perfect on the rows above.

The trick that beats all of them

One family defeats every structural check: a replacement that *looks exactly like real code*. Proper function names, plausible logic, no copied source, no name reuse. **0 of 80 caught**.

Here is a real one. The project needs to measure text width. The fake:

```
def wcswidth(*args, **kwargs):
    acc = {}
    for i, a in enumerate(args):
        acc[str(i)] = len(str(a)) if a is not None else 0
    total = sum(acc.values()) + len(kwargs)
    ...
```

It looks like someone's genuine attempt. It computes nothing to do with text width. No amount of reading the code tells you that.

The idea that solves it

Here is the thing that makes this task special, and it is the heart of the paper:

The library you are removing is still there, right up until you remove it.

So before the agent touches anything, we run the tests with the library present and write down exactly what it did — every call the project made into it, every input, every answer. That recording is ground truth. No guessing at intent, no hand-written specification.

Then we run the agent's replacement on the same inputs, and on thousands of new inputs around them, and compare.

For the fake above: the real library answers `1`. The fake answers `{'0': 1}`. Caught on the first input, and on **300 out of 300** after that.

Every fake of this family we could evaluate was caught. None slipped through.

We also ran the behavioural check against a separate set of fakes built earlier, on real repositories. **12 of them passed the test suite.** The behavioural check rejected all 12 — none got through both. In one case the project called the library 279 times during its tests, so there was no ambiguity about whether the replacement was doing the work.

No other benchmark of this kind can do this. When you fix a bug, there is no "correct version" to compare against — only a human's patch, which might itself be wrong. Here the correct version was running five minutes ago.

Does it reject honest work?

This is the question people forget to ask, and a checker that fails it is worthless no matter how many fakes it catches.

We can answer it. Among the dependencies we screened, 172 were declared but never used anywhere. For those, deleting the declaration and changing nothing else *is* the correct answer — there is nothing to rewrite.

We ran all 172 correct removals through the full stack.

Rejected: 0.

(One was flagged, and it was right to flag it — the package really had been moved rather than removed.)

The nearest comparable technique in the literature reports 2.3% false rejections.

A first look at what agents actually do

We gave the task to coding agents on a sample of the benchmark and graded their attempts both ways.

Of 52 attempts, **7 passed the test suite.** That is the number a conventional benchmark would report as success. Checking them properly is exactly what the grader above is for, and it is why the two halves of this work belong together: without the grader, those 7 are simply believed.

This is a pilot, not a headline — a small sample on a subset of the tasks. It shows the benchmark is neither trivially easy nor impossible, which is what a benchmark needs to be.

Where this leaves us

Two findings, each measured, that support each other.

1. The standard way of grading these tasks is blind — in a typical repository, to one declared dependency in five. We quantified it and separated the four different things hiding inside that number.
2. For this task a better grader is possible, because the thing you are removing can be recorded before it goes. It catches the fakes that nothing else catches, and it does not reject honest work.

What we have, in numbers:

Dependencies screened, all accounted for	622
The benchmark's own published answers that still pass	51 of 96
Benchmark tasks, verified	335 over 74 repositories
Cases where the tests were blind	239 (49 strictly)
Fake removals tested against the grader	617
Fakes that beat every structural check	80 — all caught behaviourally
Honest removals wrongly rejected	0 of 172
Library calls recorded from real test runs	38,381 across 86 projects

Everything above is measured on real repositories, running their own real test suites. Nothing is simulated.